

Instruction Set Classification: RISC vs CISC

1. What is an Instruction Set?

An **Instruction Set Architecture (ISA)** defines the set of machine-level instructions that a CPU can execute. Based on ISA design, processors are broadly classified into:

- **CISC** (Complex Instruction Set Computer)
- **RISC** (Reduced Instruction Set Computer)

2. CISC: Complex Instruction Set Computer

- Large number of instruction types and addressing modes.
- Variable instruction sizes.
- Instructions may perform multiple operations (e.g., load, operate, store).
- Often takes multiple cycles per instruction.

Example CISC Architectures

- Intel x86
- VAX
- Motorola 680x0

3. RISC: Reduced Instruction Set Computer

- Smaller, simpler set of instructions.
- Fixed instruction size (usually 32 bits).
- Load/store architecture — only load/store instructions access memory.
- Focus on pipeline-friendly, single-cycle instruction execution.
- Greater number of general-purpose registers.

Example RISC Architectures

- MIPS
- DEC Alpha
- ARM64
- Sun SPARC
- IBM 801

4. RISC vs CISC Comparison

Feature	CISC	RISC
Instruction Length	Variable	Fixed (e.g., 32 bits)
Instruction Format	Many formats	Few, regular formats
Memory Access	Operands can be memory locations	Load/store only
Operations	Complex (multi-step)	Simple (single-step)
Execution Time	Multiple cycles per instruction	Mostly single-cycle execution
Code Size	Smaller due to complex instructions	Larger, but more optimized for pipeline
Design Complexity	High	Low

5. General Instruction Format

An instruction specifies:

- The **operation** to be performed (opcode)
- The **operands** involved (registers or memory locations)

In assembly:

$$\text{op } X_1, X_2, \dots, X_n$$

Where:

- **op**: Operation code
- $X_1, X_2, \dots, X_n$: Operands

6. RISC Instruction Format (Example: RISC-I)

Key Concepts:

- Memory access only via `load` and `store` instructions.
- Arithmetic and logic operations use registers only.
- Instructions fit in a single 32-bit word.

Example: Register-to-Register Operation

$$Rd := F(Rs, S2)$$

Where:

- `Rd` – Destination register
- `Rs` – Source register
- `S2` – Second source (either register or immediate)

Note: If bit 13 is set, `S2` is interpreted as an immediate value.

7. Summary Points

- **CISC:** Flexible but complex and harder to pipeline.
- **RISC:** Simpler, faster, and more suited for pipelining and compiler optimization.
- Instruction design significantly affects CPU performance and efficiency.